Monitoring, Logging & Alerts (Observability)

6.1 Overview

The observability stack is the central nervous system of our DevOps platform. It provides real-time insights into application performance, infrastructure health, and enables rapid debugging when issues arise.

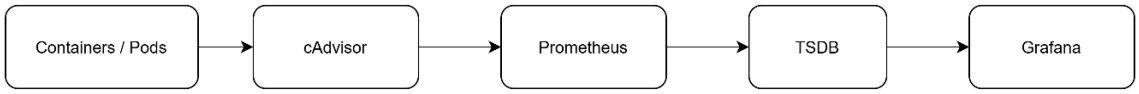
We use a best-of-breed open-source stack, all deployed inside our Kubernetes cluster:

Tool	Purpose
Prometheus	Collects and stores time-series metrics
Grafana	Visualizes metrics and logs via dashboards
Loki	Aggregates and indexes logs
Promtail	Collects logs from pods and ships to Loki

6.2 Metrics with Prometheus & Grafana

6.2.1 Prometheus Setup

Prometheus is our core metrics collection engine. It dynamically discovers and scrapes metrics from all pods and nodes inside the cluster using Kubernetes service discovery.



Installation

Step 1: Add the Helm repository

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts

helm repo update

Step 2: Create a dedicated namespace for all monitoring tools

kubectl create namespace monitoring

Step 3: Install the stack

helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack \

--namespace monitoring \

--set prometheus.ingress.enabled=true \

--set prometheus.ingress.hosts[0]=prometheus.gathergenie.in \

--set alertmanager.alertmanagerSpec.externalUrl=https://prometheus.gathergenie.in

Access:

- Internal (within cluster): http://prometheus-operated.monitoring.svc:9090
- External (public): https://prometheus.gathergenie.in

6.2.2. Grafana Setup

Grafana is our single pane of glass for all observability data. It connects to both Prometheus (for metrics) and Loki (for logs).

Installation Method

Grafana was installed as part of the same **kube-prometheus-stack Helm chart**, so it comes pre-configured with Prometheus as a data source automatically.

```
helm install grafana grafana/grafana --namespace monitoring

kubectl patch svc grafana -n monitoring \
-p '{"spec": {"type": "NodePort"}}'
```

You can expose the Grafana server using the node IP and node port.

```
kubectl get secret --namespace monitoring grafana \
 -o jsonpath="{.data.admin-password}" | base64 --decode
```

Data Sources Configured:

Data Source	URL	Purpose
Prometheus	http://prometheus-operated.monitoring.svc:9090	Metrics

Data Source	URL	Purpose
Loki	http://loki.monitoring.svc:3100	Logs

Access:

- External URL: https://grafana.gathergenie.in
- Login: Admin credentials stored securely in a Kubernetes Secret (auto-generated by Helm, retrieved via kubectl get secret)

```
# To retrieve the Grafana admin password
kubectl get secret --namespace monitoring kube-prometheus-stack-grafana \
  -o jsonpath="{.data.admin-password}" | base64 –decode
```

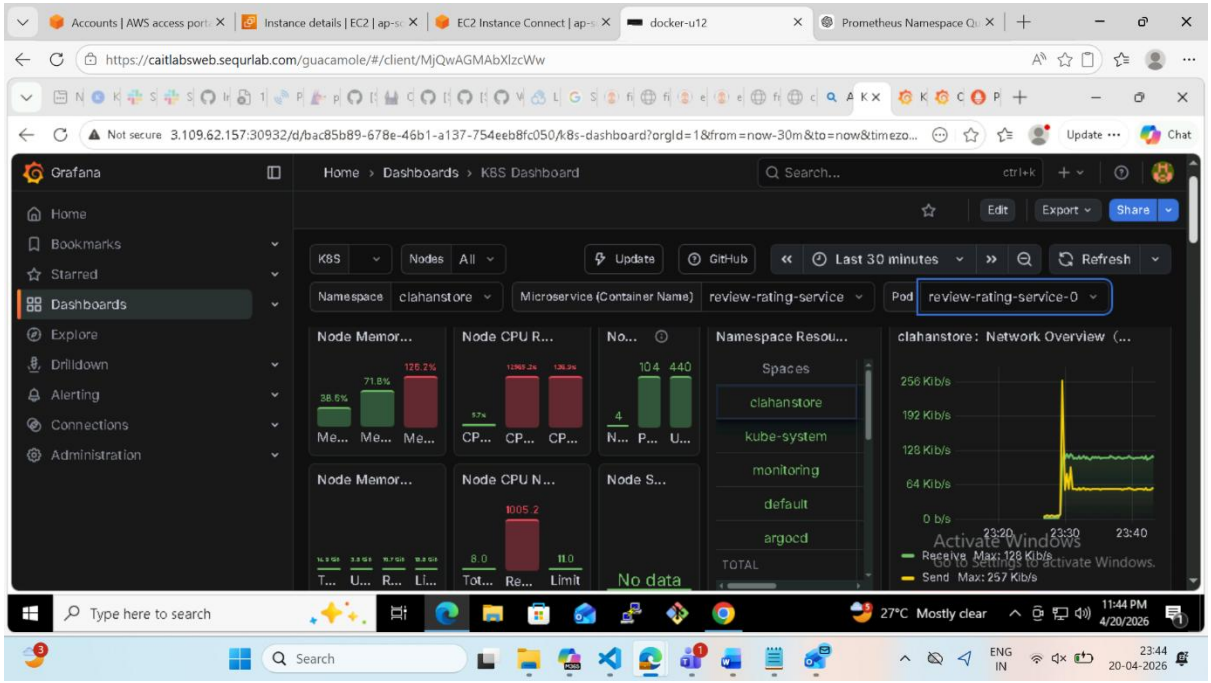
6.2.3. Key Dashboards

We use the following dashboards imported from the official Grafana dashboard library and customized for our stack.

Dashboard 1: Kubernetes Cluster Overview

(Dashboard ID: 315 from grafana.com)

Purpose: High-level bird's-eye view of the entire cluster health.



Key Panels:

Panel	Metric Used	What It Shows
Cluster CPU Usage	rate(container_cpu_usage_seconds_total[5m])	Total CPU consumed across all pods
Cluster Memory Usage	container_memory_working_set_bytes	RAM usage per namespace
Pod Status Overview	kube_pod_status_phase	Running / Pending / Failed pod counts
Node Status	kube_node_status_condition	Healthy vs NotReady nodes
Total Restarts	kube_pod_container_status_restarts_total	Crash loops indicator

Dashboard 2: Node Exporter - System Metrics

(Dashboard ID: 1860 from grafana.com)

Purpose: Low-level host/node metrics like CPU, memory, disk, and network per node.

Key Panels:

Panel	What It Shows
CPU Usage per Node	Per-core utilization
Memory Available	Free vs used memory
Disk I/O	Read/write throughput
Network Traffic	Bytes in/out per interface

Dashboard 3: Application HTTP Metrics

(Custom dashboard using kube-state-metrics and metrics-server data)

Purpose: Monitor the health of our application services at the HTTP level.

Key Panels:

Panel	Metric	What It Shows
Request Rate	rate(http_requests_total[5m])	Requests per second per service
Error Rate	rate(http_requests_total{status=~"5.."}[5m])	5xx errors per service
P95 Latency	histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))	95th percentile response time

Panel	Metric	What It Shows
Pod Restart Count	kube_pod_container_status_restarts_total	Stability indicator

6.3. Logging with Loki & Promtail

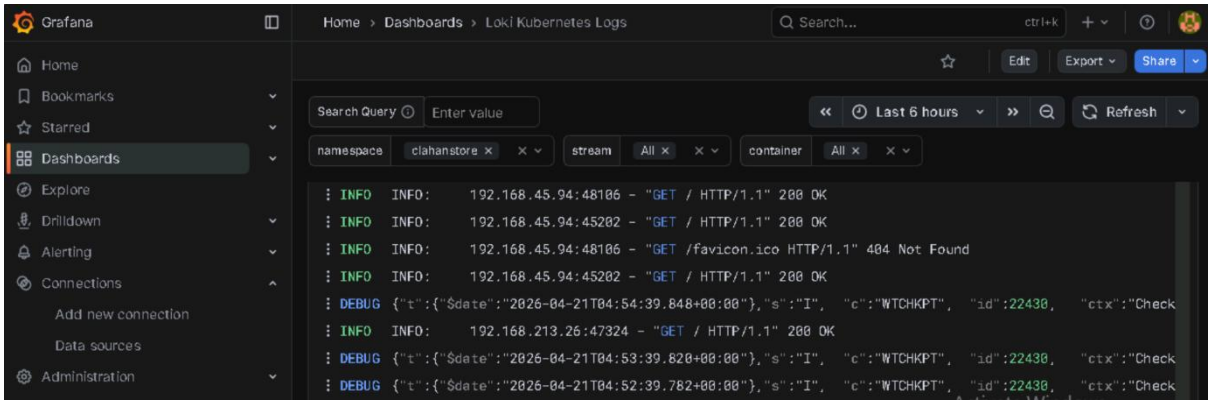
6.3.1. Promtail Setup (Log Collector)

Promtail runs as a **DaemonSet** — meaning exactly one Promtail pod runs on every worker node. It automatically finds and tails all container log files from the node's filesystem and ships them to Loki with enriched labels.

Installation:

```
# Add Grafana Helm repo
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update

# Install Promtail
helm install promtail grafana/promtail \
  --namespace monitoring \
  --set config.lokiAddress=http://loki.monitoring.svc:3100/loki/api/v1/push
```



6.3.2. Loki Setup (Log Aggregation)

Loki receives logs from Promtail, indexes the labels (not the full log content, which keeps it fast and cheap), and stores compressed log chunks.

Installation:

```
helm install loki grafana/loki \
  --namespace monitoring \
  --set persistence.enabled=true \
  --set persistence.size=2Gi
```

Access:

Logs are NOT accessed via Loki's API directly. All log querying is done through **Grafana's Explore section** using LogQL queries.